

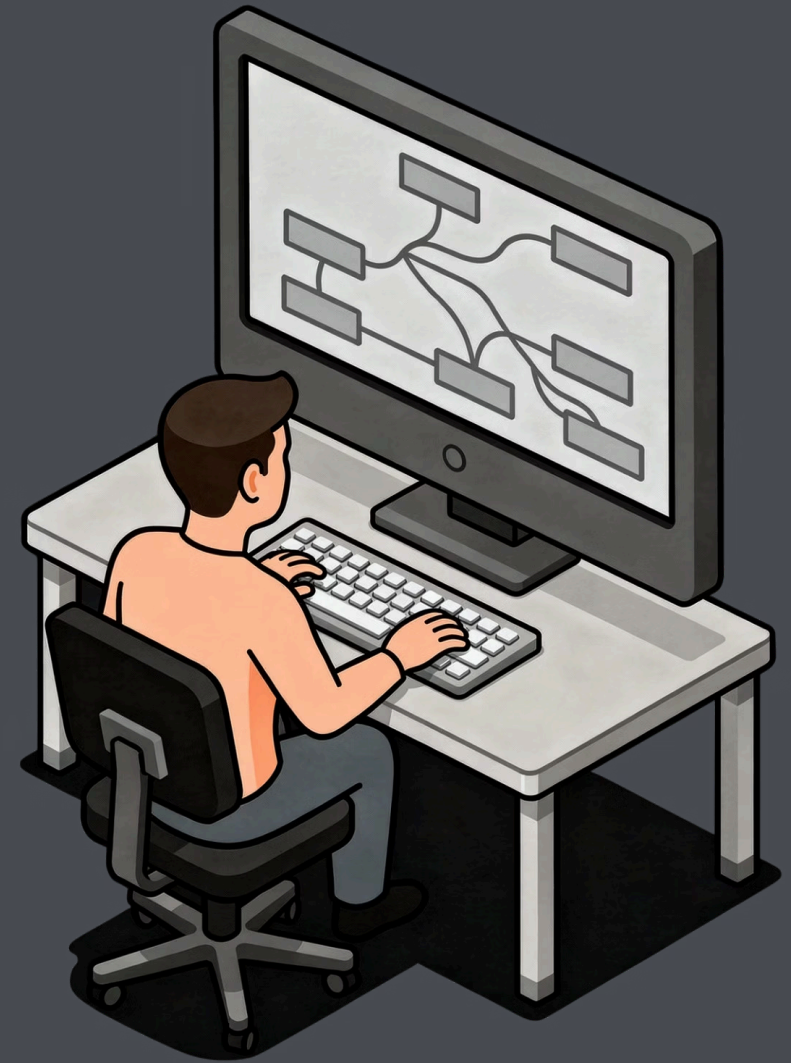
# Day 4: Advanced Git Workflows & Automation

Mastering Enterprise Collaboration & GitHub Actions

Elevating your version control skills from basic commands to managing branch strategies, resolving complex conflicts, rewriting history safely, and automating your workflow with GitHub Actions.

Sami Selim

DevOps Engineering



# What We'll Cover Today

Eight focused sections, four hands-on labs, and the tools you need to work confidently in an enterprise Git environment.

01

---

## Branching Strategies

GitFlow vs. Trunk-Based Development

03

---

## Interactive Rebase & Squash

Cleaning up messy commit history

05

---

## Advanced PRs & Git Hooks

Collaboration and shift-left automation

02

---

## Merge vs. Rebase

Golden rules and when to use each

04

---

## Cherry-Pick & Reflog

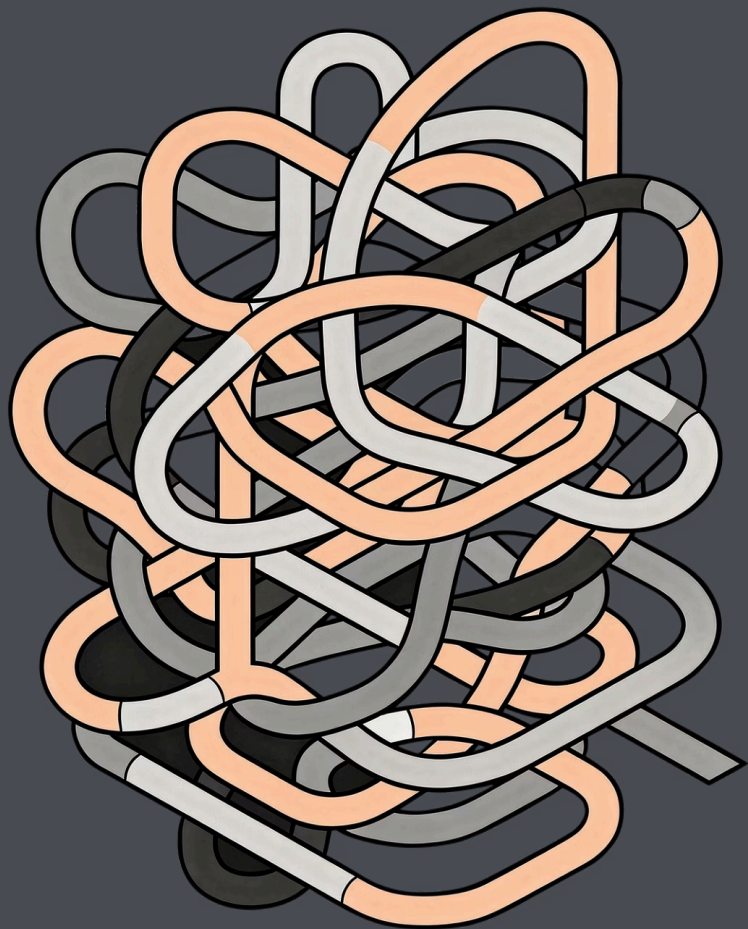
Surgical precision and disaster recovery

06

---

## GitHub Actions

CI/CD pipelines built into your repo



# The Branching Problem

In an enterprise environment, dozens of developers push changes to the same repository simultaneously. Without a clear strategy, the result is chaos — broken builds, overwritten code, and integration nightmares that cost hours to unravel.

## The Problem

Random pushes to `main` break production instantly. No coordination means constant firefighting instead of building.

## The Solution

A **Branching Strategy** defines rules: when to create branches, how to name them, and precisely when and how to merge them back.

## The Goal

Predictable integration, clean history, and a codebase that is always in a deployable state — even with 50 engineers committing daily.

# GitFlow: The Traditional Enterprise Standard

GitFlow uses explicit, long-lived branches to represent the exact state of the codebase at every phase of the release lifecycle. It was the dominant enterprise standard for over a decade.

## main / master

Strictly represents what is **running in Production**. Only hotfixes and release branches may merge here.

## develop

The **integration branch** for all upcoming features. Always one step ahead of production.

## Feature Branches

Created from `develop`, merged back into `develop` when complete. Isolated per feature.

## Release Branches

Branched from `develop` to stabilize and test a specific version before shipping to production.

## Hotfix Branches

Branched **directly from main** to patch production emergencies. Merged back into both `main` and `develop`.

# GitFlow: Pros & Cons

## ✓ Advantages

GitFlow shines in environments with strict versioned release schedules — think packaged software shipping v1.2, v1.3.

- Extremely structured and predictable
- Every branch has a clear, defined purpose
- Release and hotfix flows are well-understood by enterprise teams
- Excellent audit trail across environments

## ✗ Disadvantages

For teams shipping continuously, GitFlow becomes a bottleneck rather than a safety net.

- **Integration Hell:** Feature branches live for weeks. When they finally merge to `develop`, conflicts are catastrophic
- Long feedback loops slow down CI/CD pipelines
- Overhead of managing multiple long-lived branches adds cognitive load
- Not suited for teams deploying multiple times per day

# Trunk-Based Development

The modern DevOps standard, adopted by Google, Facebook, and most high-velocity engineering organizations. One rule governs everything.

## The Core Rule

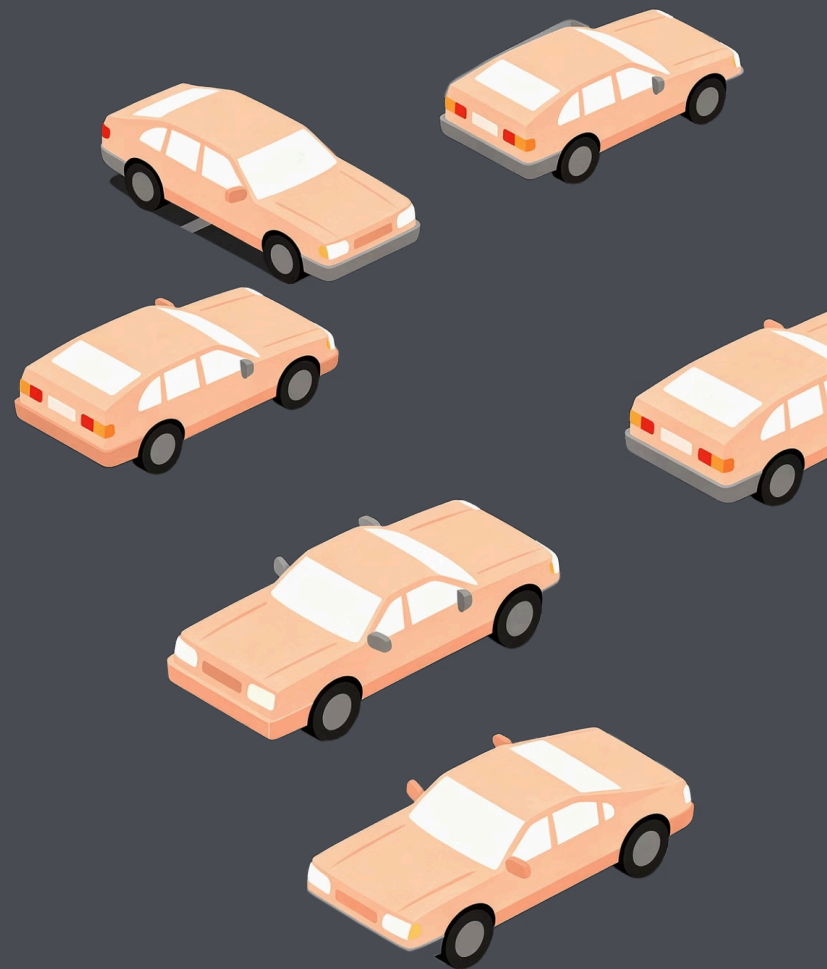
All developers merge their code directly into `main` (the "trunk") **multiple times per day**. Feature branches are short-lived — hours, not weeks.

## Continuous Integration

Automated tests run on every push. Broken code is blocked from merging before it ever touches the trunk. The pipeline is the gatekeeper.

## Feature Flags

Code ships to production but stays **hidden from users** via software toggles. Decouple deployment from release — ship safely, enable deliberately.



# GitFlow vs. Trunk-Based: Side-by-Side

| Dimension         | GitFlow                      | Trunk-Based Dev           |
|-------------------|------------------------------|---------------------------|
| Branch lifespan   | Weeks to months              | Hours to 1 day            |
| Merge frequency   | On completion                | Multiple times/day        |
| Conflict risk     | High — long divergence       | Low — small diffs         |
| CI/CD suitability | Moderate                     | Excellent                 |
| Best for          | Versioned, packaged software | SaaS, continuous delivery |
| Release mechanism | Release branches             | Feature flags + tags      |

Neither strategy is universally superior — choose based on your release cadence, team size, and deployment model.

# Keeping Your Branch Up to Date

You're working on a feature branch. Meanwhile, three teammates have merged their work into `main`. Your branch is now behind. You need their changes to avoid conflicts later — but how you bring them in matters enormously.

1

## The Situation

Your feature branch diverged from `main` 2 days ago. New commits exist on both sides.

2

## Two Choices

`git merge main` or `git rebase main` — each produces a fundamentally different history.

3

## The Stakes

The wrong choice on a shared branch can destroy teammates' local histories and require painful force-push recovery.



# git merge main

## What It Does

Creates a new **merge commit** that ties the history of both branches together at a single point in time. Both diverging lines of development are permanently preserved in the graph.

## ✓ Pros

- Completely **non-destructive** — no existing commits are altered
- Preserves the exact historical context of when branches diverged
- Safe to use on shared, public branches

## ⚠ Cons

In an active team where everyone merges constantly, the commit history becomes a dense, tangled web of merge commits — resembling a messy subway map rather than a clear narrative.

- History can become extremely difficult to read with `git log`
- Bisecting bugs in a non-linear history is painful
- Merge commits add noise without adding meaning

# git rebase main

## What It Does

Unplugs your feature branch from where it originally diverged, fast-forwards it to the **tip of main**, and reapplies your commits one by one on top — as if you started your work today.

## ✓ Pros

- Creates a clean, straight, **linear history**
- `git log` reads like a clear story from start to finish
- Easier to bisect, review, and audit

## ⚠ Cons

Rebase **rewrites history**. Every rebased commit receives an entirely new SHA hash — even if the content is identical. This is the critical trade-off that defines when rebase is safe and when it is dangerous.

- New commit hashes break shared branch references
- Force-pushing to a shared branch causes chaos for teammates
- Must only be used on private, local branches

# The Golden Rule of Rebase

**NEVER rebase commits that exist outside your local repository and that other people may have based their work on.**

When you rebase a shared branch like `main` or `develop`, you generate new commit hashes for every existing commit. Everyone else's local clone still references the old hashes — their histories are now incompatible with origin. The recovery process is painful and disruptive for the entire team.

## ✓ Rebase: Local Private Branches

Your feature branch that only exists on your machine. Rebase freely to clean up history before opening a PR.

## ✓ Merge: Shared Public Branches

`main`, `develop`, or any branch your teammates have checked out. Always use merge here.

# Interactive Rebase: Cleaning Up Messy Work

In the real world, a developer's working commits often look like: "WIP", "fixed typo", "actually fixed it", "tests pass now". These commits are useful while working, but they pollute the shared timeline and make code review painful.

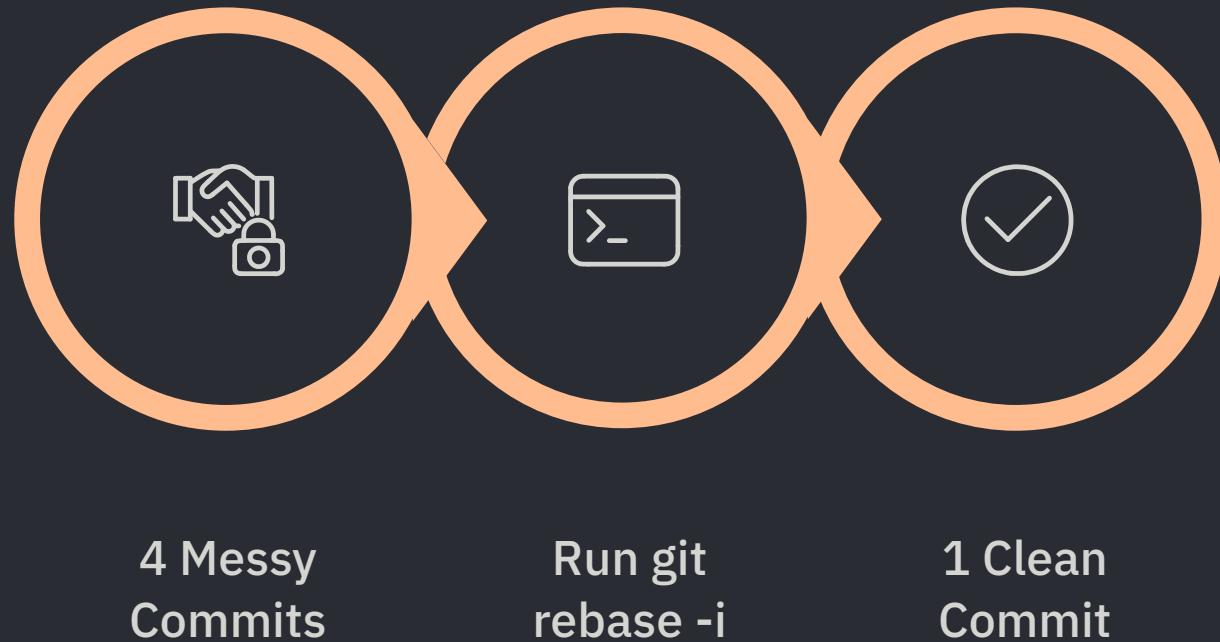
`git rebase -i HEAD~4` opens an interactive editor letting you manipulate the last 4 commits before they ever leave your machine. You can reorder, edit, delete, or combine them — giving your teammates a polished, intentional commit history instead of a stream of consciousness.

 Interactive rebase is one of the most powerful — and most underused — tools in a senior engineer's toolkit. Master it before your next PR.



# Squashing Commits

The most common interactive rebase operation is **squash**. Change a commit's instruction from `pick` to `squash` (or `s`) and Git melds that commit into the one before it, prompting you to write a single unified commit message.



The result: 10 messy work-in-progress commits collapse into a single, atomic, meaningful commit like `feat: add user auth` — exactly what reviewers and future maintainers want to see in the log.

# Lab 1: Rebase & Squash

Objective: Clean up a messy branch history before sharing it with your team.

1

## Create Messy Commits

Create a new branch and add 4 commits with throwaway messages: "WIP 1", "WIP 2", "Typo fix", "Tests pass".

2

## Open Interactive Rebase

Run `git rebase -i HEAD~4`. Your editor opens showing all 4 commits with the `pick` instruction.

3

## Mark Commits to Squash

Keep the first commit as `pick`. Change the next 3 to `squash`. Save and exit the editor.

4

## Write a Clean Message

Git prompts for a unified commit message. Write something meaningful like `feat: implement login feature`.

5

## Verify the Result

Run `git log --oneline`. Confirm 4 messy commits have become 1 clean, descriptive commit.



# Cherry-Picking: Surgery on the Commit Graph

Sometimes you need *one specific commit* from another branch — not the entire branch, just that single change. This is exactly what `git cherry-pick` was designed for.

## The Scenario

A critical security patch was committed to `develop` as commit `e43a6f2`. You need it on `main` right now — you can't wait for the next release cycle.

## The Command

`git cherry-pick e43a6f2` plucks that exact commit snapshot and applies it as a new commit on your current branch. The original commit remains untouched on `develop`.

## When to Use It

Hot fixes that must ship immediately, backporting features to older release branches, and selectively applying patches across multiple long-lived branches.

# The Reflog: Your Ultimate Safety Net

## The Disaster

You typed `git reset --hard HEAD~5` and wiped out the last two days of commits. They don't appear in `git log`. Panic sets in.

## The Reality

In Git, almost nothing is truly deleted. The commits still exist in the object store — they've just lost their references. Until garbage collection runs (typically after 30 days), they're fully recoverable.

## `git reflog` to the Rescue

The **Reference Log** records every movement of the HEAD pointer — every checkout, reset, rebase, commit, and merge — even for commits with no branch pointing to them.

- Run `git reflog` to see the full timeline
- Find the hash of the commit you lost
- Run `git reset --hard <recovered-hash>`
- Your work is fully restored

✓ Reflog is local only — it doesn't sync to GitHub. Keep this in mind for remote recovery scenarios.



## LAB 2

# Lab 2: Cherry-Picking & Reflog Recovery

Objective: Perform surgical commit transplants and recover deliberately lost code.

1

## Create a Commit to Transplant

Create a feature branch, make a meaningful commit, and note its full hash from `git log --oneline`.

2

## Cherry-Pick to main

Switch back to `main` and run `git cherry-pick <hash>`. Verify the commit appears on `main` with `git log`.

3

## Deliberately Destroy a Commit

Run `git reset --hard HEAD~1` to wipe the most recent commit. Confirm it is gone from `git log`.

4

## Recover with Reflog

Run `git reflog` to find the lost commit hash. Run `git reset --hard <hash>` to fully restore it.

# Resolving Merge Conflicts

Merge conflicts are inevitable in any collaborative codebase. They occur when two branches modify the **exact same lines** in the same file, and Git cannot determine which version is correct. This requires a human decision.

<<<<<< HEAD

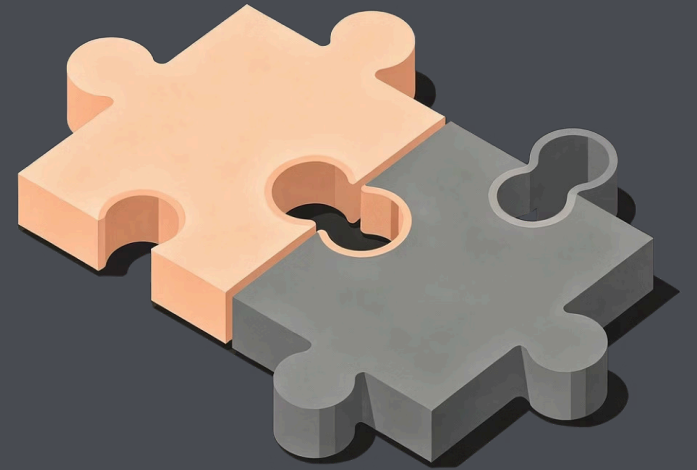
Everything between this marker and ===== is the content from **your current branch** (the branch you ran the merge or rebase on).

===== Divider

The separator between the two conflicting versions. Lines above are "ours," lines below are "theirs."

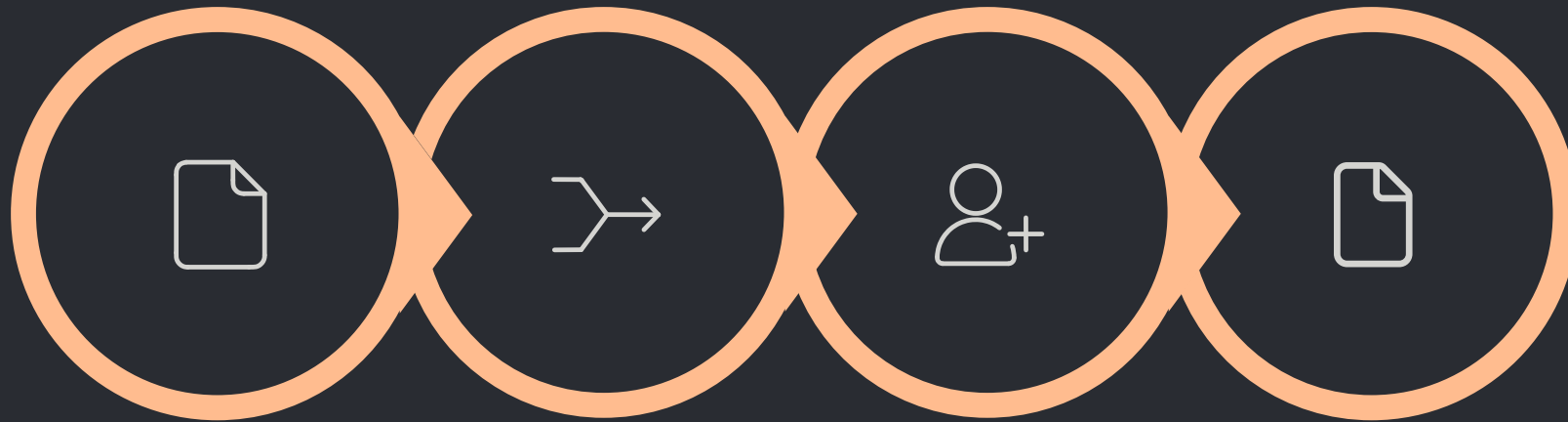
>>>>>> branch-name

Everything between ===== and this marker is the content from the **incoming branch** being merged in.



# Resolving Conflicts: The Process

Conflicts are not emergencies — they're just textual differences waiting for a human decision. A calm, methodical approach resolves them in minutes.



Open File

Edit Conflicts

Stage File

Complete  
Commit

## After Merging

Fix the file, run `git add <file>`, then `git commit`. Git auto-generates the merge commit message.

## After Rebasing

Fix the file, run `git add <file>`, then `git rebase --continue`. Repeat for each conflicting commit.

## Abort if Needed

Use `git merge --abort` or `git rebase --abort` to fully cancel and return to the pre-operation state.

# Lab 3: The Merge Conflict

Objective: Deliberately create and manually resolve a merge conflict end-to-end.

1

## Create the First Version

On `main`, edit line 1 of `file.txt` to say "Apples". Stage and commit the change.

2

## Create a Competing Branch

Create `branch-b`. Edit the same line 1 to say "Oranges". Commit on `branch-b`.

3

## Create a Third Version on main

Check out `main`. Edit line 1 to say "Bananas". Commit. Now both branches have diverged on the same line.

4

## Trigger the Conflict

Run `git merge branch-b`. Git outputs a `CONFLICT` warning and halts. Open `file.txt` to see the conflict markers.

5

## Resolve and Finalize

Edit the file to your preferred resolution, remove all conflict markers, run `git add file.txt` and `git commit`.

# GitHub & the Pull Request Model

While Git manages version history locally, **GitHub drives the collaborative workflow**. The Pull Request is the central mechanism for peer review, discussion, and controlled integration in enterprise teams.



## Unified Code Diff

Every PR provides a side-by-side view of exactly what changed. Reviewers see insertions, deletions, and the full file context — no guessing what shipped.



## Line-Level Discussion

Reviewers can comment on specific lines, suggest exact code changes inline, and hold threaded conversations. Every discussion is preserved in the PR history.



## Required Approvals

PRs can be blocked from merging until a minimum number of approvals are received. Combined with passing CI checks, this enforces code quality at scale.



# PR Best Practices for Enterprise Teams

## Keep PRs Small and Focused

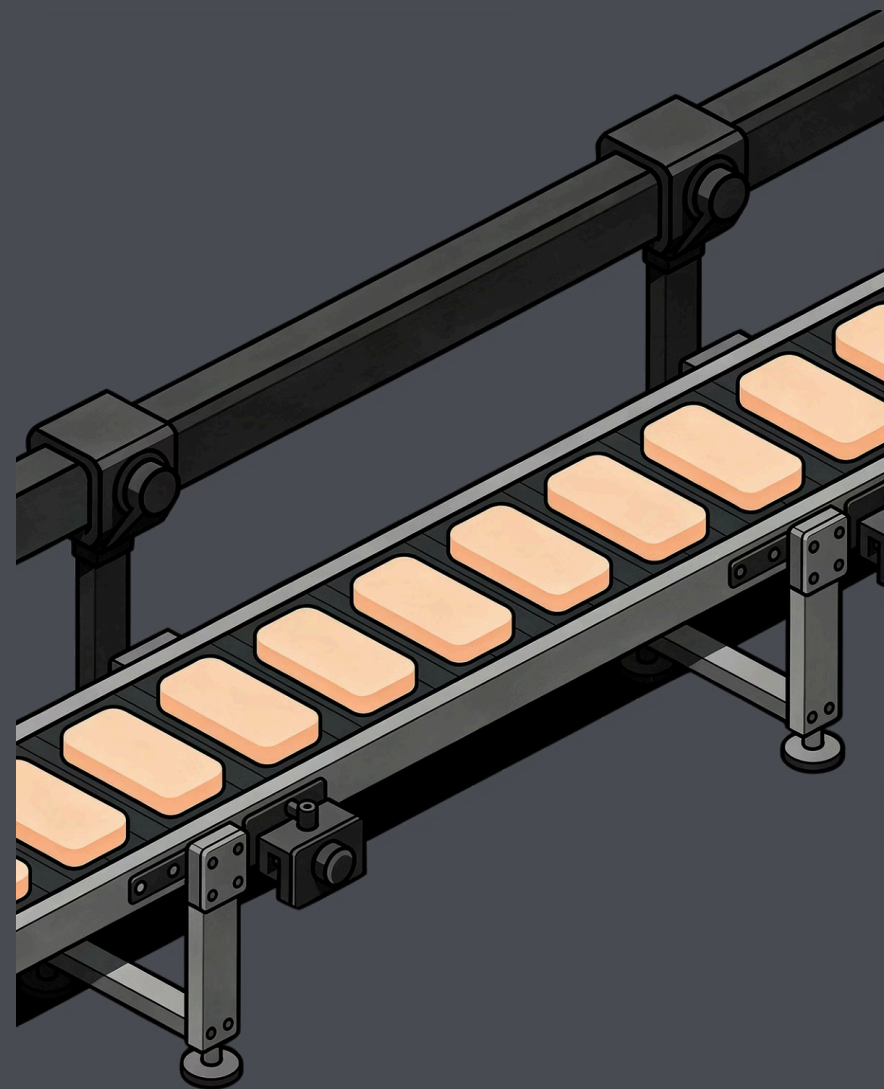
A 1,000-line PR will barely be reviewed — it's cognitively overwhelming and reviewers rubber-stamp it. A focused 50-line PR gets detailed, meaningful feedback. Aim for a single logical change per PR. Small PRs merge faster, break less, and ship sooner.

## Use Draft PRs Early

Open a Draft PR as soon as you start working. This signals work-in-progress to the team, prevents duplication of effort, and enables early architectural feedback before you've invested hours in the wrong direction.

## Enforce Branch Protection Rules

Configure GitHub to block any code from merging to `main` without at least 1 approval and all required CI checks passing. This makes quality enforcement automatic rather than relying on human memory.



# Introduction to CI/CD

## Continuous Integration (CI)

Automatically **building and testing** the codebase every time a developer pushes changes. Every push triggers the pipeline. Broken builds are caught immediately, before they compound.

## Continuous Delivery/Deployment (CD)

Automatically **deploying** tested code to staging or production environments after CI passes. Removes the manual, error-prone release ceremony entirely.

The combined goal: catch bugs early while they're cheap to fix, automate repetitive toil so engineers focus on meaningful work, and ship updates faster and more safely than any manual process allows.

# What Are GitHub Actions?

## Native CI/CD Built Into GitHub

GitHub Actions is a **first-class CI/CD platform built directly into GitHub** — no external Jenkins server, no CircleCI subscription required. It allows you to define automated workflows triggered by any GitHub event: a push, a PR opened, an issue labeled, a release published, or a scheduled cron job.

Workflows are defined as **YAML files** stored in `.github/workflows/` within your repository. They're versioned alongside your code, reviewed in PRs, and visible to every team member.

## Key Capabilities

- Run on GitHub-hosted or self-hosted runners
- Triggered by 30+ GitHub event types
- Access to thousands of community-built Actions in the Marketplace
- Secure secret management built in
- Matrix builds across multiple OS and language versions
- Free minutes included for public repositories



# GitHub Actions: Core Terminology

Before writing your first workflow, understand the five building blocks that every Actions configuration is composed of.



## Workflow

A configurable automated process defined in a single `.yaml` file. Example: *"Test and Build on every push."*



## Event

The GitHub trigger that starts the workflow. Examples: `on: push`, `on: pull_request`, `on: schedule`.



## Job

A set of steps that execute on the **same runner**. Jobs run in parallel by default unless dependencies are declared.



## Step

A single task within a job — either a shell command (`run:`) or a pre-built Action (`uses:`).



## Runner

The virtual machine executing the job. Most commonly `ubuntu-latest`, but Windows and macOS runners are also available.

# A Simple Workflow: Node.js CI

A real, working GitHub Actions workflow for a Node.js project. Every push triggers an automated test run on a fresh Ubuntu environment.

```
name: Node.js CI

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Use Node.js 16.x
        uses: actions/setup-node@v3
        with:
          node-version: '16.x'

      - run: npm install
      - run: npm test
```

## actions/checkout@v3

A pre-built Action from the Marketplace that clones your repository onto the runner. Nearly every workflow starts with this step.

## actions/setup-node@v3

Installs and configures the specified Node.js version. The `with:` block passes configuration parameters to the Action.

# Advanced GitHub Actions Features



## Matrix Builds

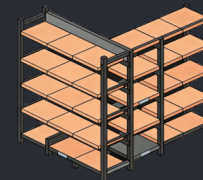
Run your test suite simultaneously across **multiple language versions and operating systems** in a single workflow definition. Catch compatibility issues before users do.

```
strategy:
  matrix:
    node: [14, 16, 18]
    os: [ubuntu, windows]
```



## Secrets Management

Store API keys, credentials, and tokens safely in **GitHub Settings → Secrets**. Access them in your workflow via `${{ secrets.API_KEY }}` — they're masked in all logs and never exposed in plain text.



## Actions Marketplace

Thousands of **pre-built, community-maintained Actions** are available — deploy to AWS, send Slack notifications, publish Docker images, post to Jira, and more. Search at [github.com/marketplace](https://github.com/marketplace).

# Lab 4: Your First GitHub Actions Workflow

Objective: Automate Python testing using GitHub Actions — from zero to a passing CI pipeline.

1

## Set Up the Repository

Create a new GitHub repository and clone it locally. This will be the home of your first automated pipeline.

2

## Write a Python App & Test

Create `app.py` with a simple function (e.g., `add(a, b)`) and `test_app.py` with a `pytest` assertion against it.

3

## Create the Workflow File

Create the folder `.github/workflows/` and add `python-app.yml`. This is where the automation is defined.

4

## Configure the YAML

Set on: `push`, use `ubuntu-latest` runner, checkout code, setup Python with `actions/setup-python`, install dependencies, and run `pytest`.

5

## Push & Watch It Run

Push to GitHub. Navigate to the **Actions tab** and watch your workflow execute in real time. Green checkmark = success.

# Chapter Recap: GitHub Actions in Practice

With GitHub Actions, the entire CI/CD loop lives inside your repository. No external infrastructure, no separate configuration system — just YAML files versioned alongside your code.

## Every Push is Validated

Automated tests run on every push or PR. Broken code is caught before it ever reaches a reviewer's screen.

## Secrets Stay Secure

Credentials never appear in code. GitHub's encrypted secret store keeps sensitive values out of logs and out of commits.

## Scale with Matrix Builds

Test across every supported environment in parallel. Catch regressions on Windows that only a Mac developer would have missed.

## Extend with the Marketplace

Thousands of maintained Actions handle deployment, notifications, code scanning, and infrastructure updates — no custom scripting required.

# The Four Hands-On Labs: A Quick Reference

| Lab   | Topic                | What You Practice                                                                      | Key Commands                                           |
|-------|----------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------|
| Lab 1 | Rebase & Squash      | Transform 4 messy WIP commits into 1 clean, atomic commit with a proper message        | <code>git rebase -i HEAD~4</code>                      |
| Lab 2 | Cherry-Pick & Reflog | Transplant a single commit across branches and recover a deliberately destroyed commit | <code>git cherry-pick</code> , <code>git reflog</code> |
| Lab 3 | Merge Conflict       | Create a real conflict on the same line across two branches and resolve it manually    | <code>git merge</code> , conflict markers              |
| Lab 4 | GitHub Actions       | Write a YAML workflow that automatically runs pytest on every push to GitHub           | <code>.github/workflows/*.yml</code>                   |

# Day 4 Summary: Advanced Git Workflows



## Branching Strategies

GitFlow for versioned releases; Trunk-Based Development for continuous delivery at scale

→ Trunk-Based Development favors speed and CI over heavyweight branching

→ `git reflog` is your local safety net against any destructive command



## Golden Rebase Rule

Rebase local private branches only. Merge shared public branches. Never rewrite shared history.

→ Interactive rebase and squash produce clean, reviewable commit history before every PR

→ GitHub Actions automates tests and deployments directly within your repository — no external tooling required



## Labs Completed

Squash, Cherry-Pick, Conflict Resolution, GitHub Actions — each reinforcing production-grade habits



# What Separates Junior from Senior Engineers

**Mastery of Git is what separates Junior developers from Senior engineers.** The commands are the same — the judgment about when and how to use them is what matures with experience.

Junior engineers commit and push. Senior engineers think about the shape of history they're creating: Is this commit atomic? Will this PR be reviewable? Is my branch safe to rebase? Every interaction with Git is an opportunity to communicate intent clearly to your future teammates — and to your future self.



# CI/CD: The Backbone of Modern DevOps

## Why CI/CD Matters

Organizations that practice continuous integration deploy code **orders of magnitude more frequently** than those that don't, with a fraction of the change failure rate. CI/CD is not an optional optimization — it is the foundation of competitive engineering at scale.

## GitHub Actions: The Ideal Starting Point

For teams already on GitHub, Actions removes every barrier to entry:

- No infrastructure to provision or maintain
- Workflows live in the same repo as the code
- Reviewed and versioned alongside features
- Free for public repositories, generous free tier for private
- Marketplace Actions accelerate common tasks to minutes

✓ The best CI/CD pipeline is the one your team will actually use and maintain. Start simple, then grow.

# Commit Message Conventions: The Missing Layer

Squashing and rebasing give you the opportunity to write excellent commit messages. The **Conventional Commits** standard is widely adopted in enterprise teams and works seamlessly with automated changelog generation.

| Prefix    | Use For                             | Example                                   |
|-----------|-------------------------------------|-------------------------------------------|
| feat:     | A new feature                       | feat: add OAuth2 login support            |
| fix:      | A bug fix                           | fix: resolve null pointer in user service |
| chore:    | Build, tooling, dependencies        | chore: upgrade jest to v29                |
| refactor: | Code change without behavior change | refactor: extract auth middleware         |
| docs:     | Documentation only                  | docs: update API authentication guide     |
| test:     | Adding or updating tests            | test: add coverage for checkout flow      |
| ci:       | CI/CD configuration changes         | ci: add matrix build for Node 18          |

# Branch Protection Rules: Enforcing Quality at Scale

Individual discipline is not a scalable quality strategy. GitHub's **Branch Protection Rules** make quality enforcement automatic and non-negotiable — no one, including repository administrators, can bypass them without an explicit override.

## Require Pull Request Reviews

Set a minimum number of required approving reviews (typically 1–2) before any branch can merge to `main`. Optionally require review from designated Code Owners.

## Require Status Checks

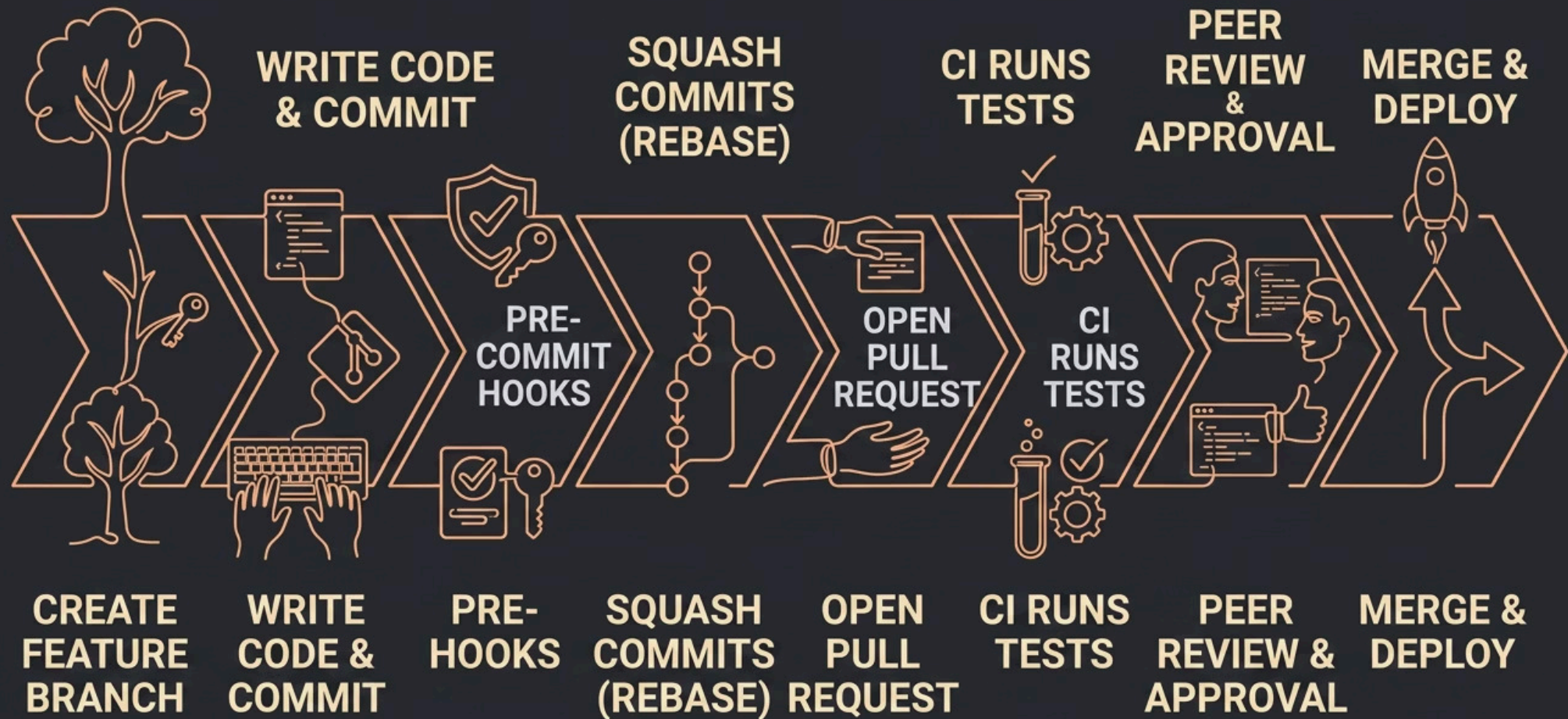
Block merges unless specific CI jobs (tests, linting, security scans) have passed. The pipeline becomes the final gatekeeper, not the merge button.

## Restrict Force Pushes

Disable `git push --force` on protected branches entirely. Prevents history rewriting accidents that corrupt teammates' local repositories.



# The Developer Workflow: End-to-End



This end-to-end workflow integrates every concept from today's session into a single, repeatable loop that scales from a two-person startup to a 500-engineer enterprise organization.

# Next Steps & Continued Practice

## Daily Rebase Practice

Incorporate `git rebase -i` into your daily routine. Before every PR, clean up your commit history. It becomes second nature within a week.

## Deploy with Actions

Write a GitHub Action that deploys a static HTML site to GitHub Pages. This adds the CD half of the CI/CD equation to your skill set with minimal infrastructure complexity.

## Explore Infrastructure as Code

Prepare for the upcoming AWS modules by studying Terraform and CloudFormation basics. IaC extends the same "version everything" philosophy from code to infrastructure.

## Add a Pre-Commit Hook

Install Husky in a personal project and configure a `pre-commit` hook that runs ESLint or Black. Experience shift-left validation firsthand.





# Q&A

## Questions?

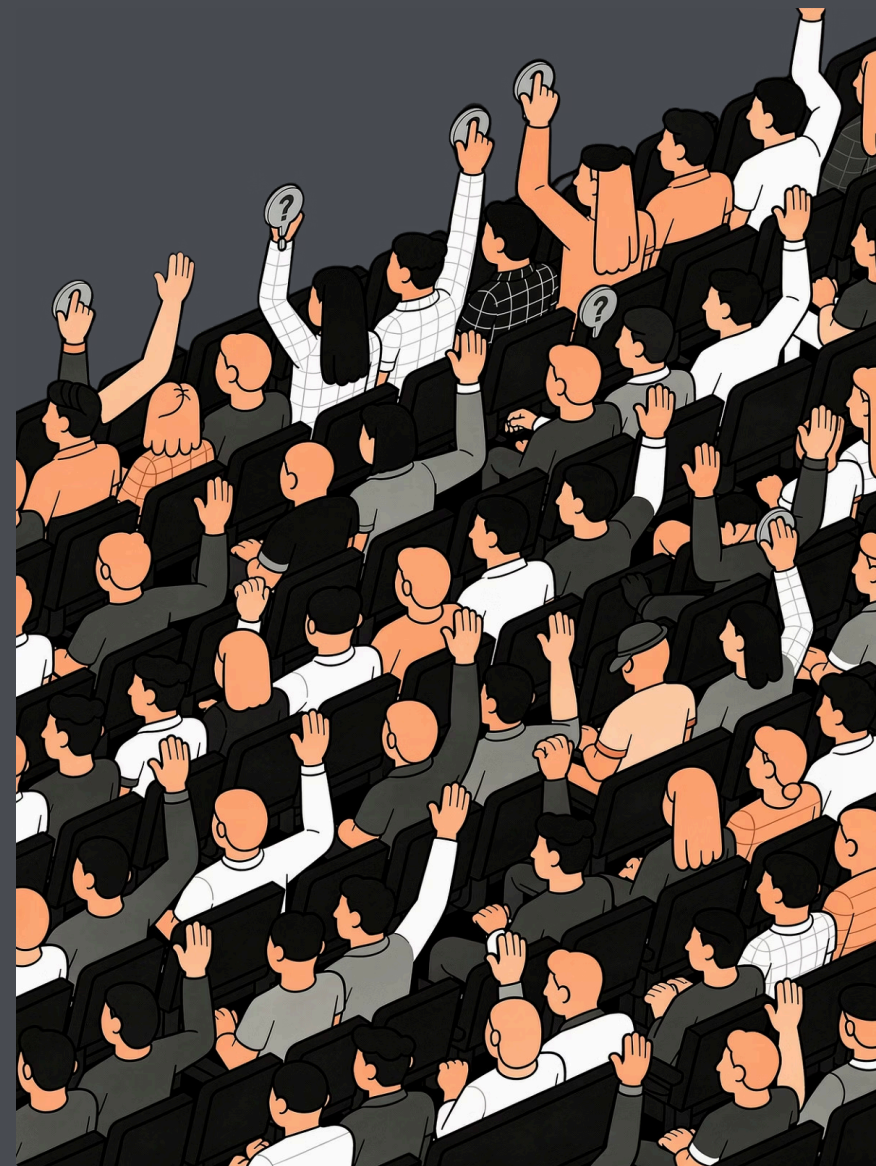
Open floor for questions on branching strategies, rebase workflows, merge conflict resolution, Git Hooks, or GitHub Actions. No question is too basic — the goal is for everyone to leave confident enough to apply these patterns in production tomorrow.

### Key Takeaway

The tools covered today — rebase, squash, cherry-pick, reflog, branch protection, and GitHub Actions — are the daily instruments of senior engineers at every leading technology organization. Practice them deliberately until they are muscle memory.

### Coming Up Next

AWS modules begin with infrastructure as code fundamentals. The same principles of automation, repeatability, and version control you've applied to Git apply equally to cloud infrastructure. See you there.



# Thank You

Sami Selim

DevOps Engineering

See you in the next session.

